TRƯỜNG ĐẠI HỌC ĐỒNG THÁP
Khoa Công nghệ và Kỹ thuật

IN4211 · CHƯƠNG 2

Chương 2: OOP nâng cao, SOLID, Design Patterns và Git

Refactoring EduHub · 9 tiết

Giảng viên **ThS. Nguyễn Tuấn Kiệt**
Năm học 2026–2027

SOLID · Patterns · xUnit · Git

2.1 Năm nguyên lý SOLID
2.2 Bốn pattern trọng tâm
2.3 Các pattern còn lại và pattern .NET
2.4 Unit test đầu tiên (xUnit)
2.5 Git: rebase, conflict, review PR

Bài tập Ch02
Refactoring EduHub

0836 980 284 nguyentuankiet@dthu.edu.vn

Nội dung Chương 2

🧱 **2.1** — Năm nguyên lý SOLID (ví dụ sai và đúng)

🏭 **2.2** — Bốn pattern trọng tâm gắn với EduHub:
Factory, Decorator, Strategy, Observer

🧩 **2.3** — Pattern còn lại; pattern có sẵn trong .NET: DI, middleware, Options

🧪 **2.4** — Unit test đầu tiên cho lớp Domain: xUnit, Arrange-Act-Assert

🌿 **2.5** — Git nâng cao: rebase, xử lý conflict, code review trên Pull Request

📝 **Thực hành** — Bài tập Ch02: Refactoring EduHub

Ôn tập Chương 1

ACTIVE RECALL Kể tên các entity của EduHub và quan hệ giữa chúng (một CLB có nhiều thành viên, nhiều sự kiện).

ACTIVE RECALL Class khác interface ở điểm nào? Khi nào dùng cái nào?

ACTIVE RECALL Vì sao các thuộc tính nên dùng *private set* và có method hành vi?

ACTIVE RECALL Quy trình Git ở Chương 1: clone, branch, commit, Pull Request. Commit message viết theo quy ước nào?

Khởi động: Abstract class và interface

🔌 Interface

- Hợp đồng: **làm được gì**, không có code làm sẵn
- Một class ký được **nhiều** interface
- Dễ thay bằng bản giả khi kiểm thử

🏗️ Abstract class

- Chia sẻ **code dùng chung** cho class con
- Chỉ kế thừa được **một** class
- Có field, constructor, method làm sẵn

Chương này dùng interface rất nhiều: **SOLID** và các **pattern** đều dựa vào việc "nói chuyện qua hợp đồng" thay vì qua class cụ thể.

Khởi động: Generic, LINQ, Nullable trong một slide

```
List<Member> members = repo.GetByClub(clubId);   // Generic: chỉ chứa Member

var names = members
    .Where(m => m.Email.EndsWith("dthu.edu.vn"))  // LINQ: lọc
    .OrderBy(m => m.Name)                          // sắp xếp
    .Select(m => m.Name)                           // đổi dạng
    .ToList();                                     // chạy thật

Club? club = clubs.GetById(id);                   // Nullable: có thể là null
var title = club?.Name ?? "(không có)";           // ?. an toàn, ?? dự phòng
```

Đoạn mã 2.1. Generic, LINQ và Nullable thường gặp trong code EduHub.

Ba công cụ này xuất hiện trong mọi ví dụ phía sau.

2.1

PHẦN 1 / 5

MỤC 2.1 CỦA ĐỀ CƯƠNG

Năm nguyên lý SOLID

- SOLID là gì và vì sao cần
- S, O, L, I, D: ví dụ sai và đúng với EduHub
- Dependency Injection
- Áp dụng vào EduHub · Lab tìm lỗi SOLID

2.1 SOLID là gì?

S **Single Responsibility** — mỗi class một việc

O **Open/Closed** — thêm mới, không sửa code cũ

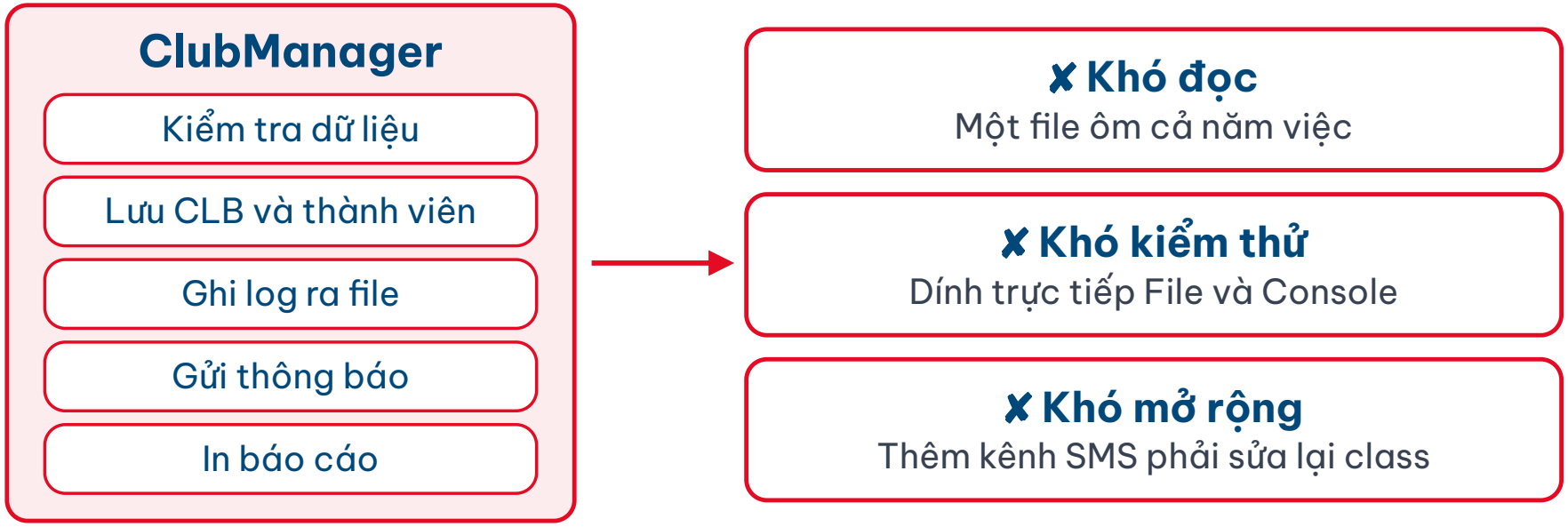
L **Liskov Substitution** — class con thay được class cha

I **Interface Segregation** — nhiều interface nhỏ

D **Dependency Inversion** — dựa vào interface, không dựa vào class cụ thể

- Năm nguyên lý của Robert C. Martin để code **dễ đọc, dễ sửa, dễ kiểm thử**
- Không phải luật cứng: là bộ câu hỏi để tự rà thiết kế của mình

Vì sao cần SOLID? Nhìn vào ClubManager



Hình 2.1. Lớp ClubManager của starter làm quá nhiều việc (God class) nên khó sửa và khó kiểm thử (tự vẽ).

Bài tập Ch02 bắt đầu từ chính class này: bạn tìm vi phạm rồi refactor.

S — Single Responsibility: mỗi class một việc



Hình 2.2. Tách ClubManager thành các class nhỏ, mỗi class chỉ có một lý do để thay đổi (tự vẽ).

"Một class chỉ nên có một lý do để thay đổi."

SRP — sai và đúng

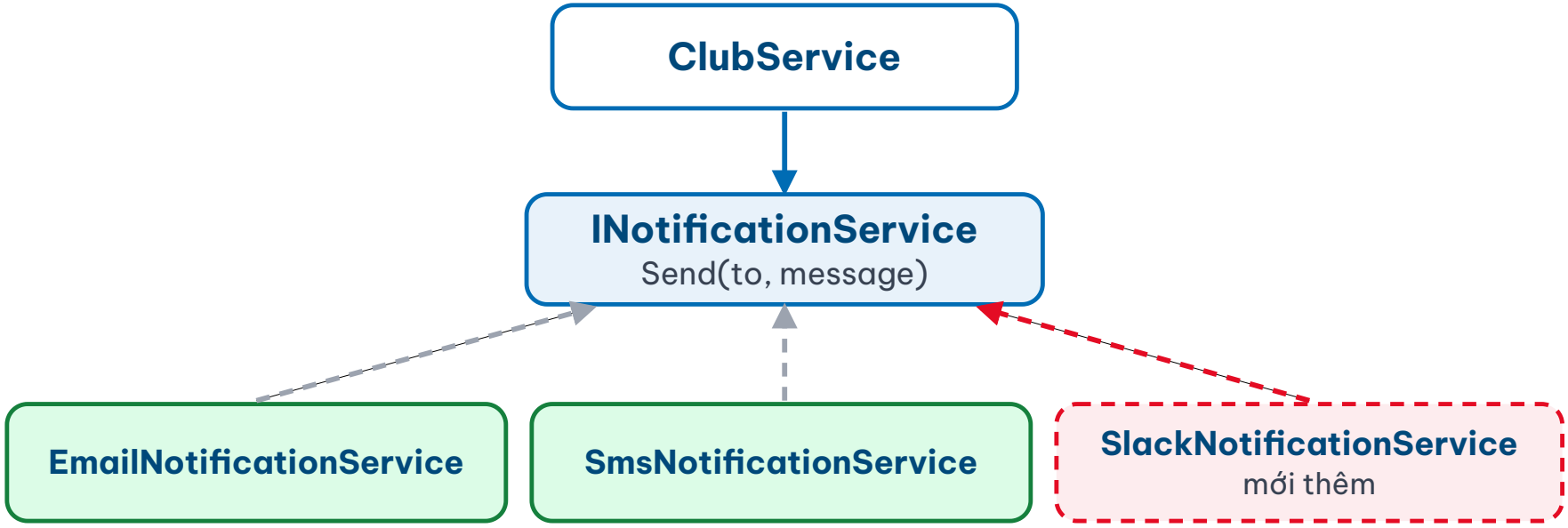
SAI — CreateClub vừa kiểm tra, vừa lưu, vừa ghi log, vừa báo tin

```
public void CreateClub(string name, string email) {
  if (string.IsNullOrWhiteSpace(name)) throw new ArgumentException();  // kiểm tra
  _clubs.Add(new Club { Name = name });                                // lưu
  File.AppendAllText("club-log.txt", $"Created: {name}");  // ghi log
  Console.WriteLine($"Sending email to {email}");          // thông báo
}
```

ĐÚNG — mỗi việc giao cho đúng chỗ, ClubService chỉ điều phối

```
var club = Club.Create(name, description);   // Domain tự kiểm tra tên
_clubs.Add(club);                             // repository lưu
_logger.Info($"Club created: {name}");        // logger ghi
_notifier.Send(presidentEmail, "Club created");  // notifier gửi
```

2.1 O — Open/Closed: thêm mới, không sửa cũ



Hình 2.3. Thêm kênh Slack chỉ là thêm một class cài đặt INotificationService; ClubService không đổi (tự vẽ).

"Mở để mở rộng, đóng để sửa đổi."

OCP — sai và đúng

SAI — thêm SMS hay Slack là phải mở method này ra sửa

```
if (email.Contains("@gmail.com"))
    Console.WriteLine($"Sending Gmail notification to {email}");
else if (email.Contains("@yahoo.com"))
    Console.WriteLine($"Sending Yahoo notification to {email}");
else
    Console.WriteLine($"Sending generic email to {email}");
```

ĐÚNG — thêm class mới, không sửa code cũ

```
public interface INotificationService { void Send(string to, string msg); }
public class EmailNotificationService : INotificationService { /* ... */ }
public class SmsNotificationService   : INotificationService { /* ... */ }  // mới
public class SlackNotificationService : INotificationService { /* ... */ }  // mới
```

L — Liskov Substitution: class con thay được class cha



Hình 2.4. Bản ReadOnly hứa có Add nhưng lại ném lỗi nên làm sập code đang dùng IClubRepository (tự vẽ).

"Chỗ nào dùng được kiểu cha thì cũng dùng được kiểu con mà không sai hành vi."

LSP — sai và đúng

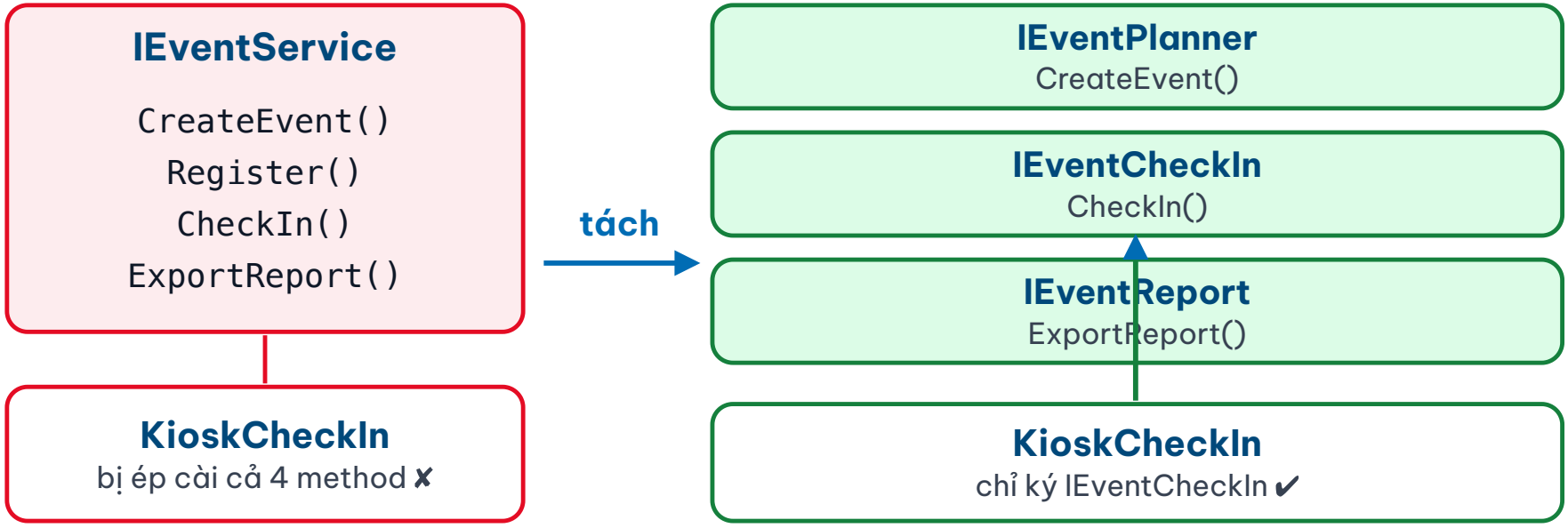
SAI — class con phá vỡ lời hứa của interface

```
public class ReadOnlyClubRepository : IClubRepository {
  public void Add(Club club) => throw new NotSupportedException();  // phá hợp
đồng
}
// Mọi nơi gọi repo.Add(club) sẽ sập khi gặp bản này
```

ĐÚNG — chỉ hứa những gì làm được

```
public interface IClubReader {
  IReadOnlyList<Club> GetAll();  Club? GetById(Guid id);
}
public interface IClubRepository : IClubReader { void Add(Club club); }
public class ReadOnlyClubRepository : IClubReader { /* chỉ đọc */ }
```

I — Interface Segregation: nhiều interface nhỏ



Hình 2.5. Tách interface "béo" thành các interface nhỏ để mỗi class chỉ ký phần mình cần (tự vẽ).

"Đừng ép class phải cài những method nó không dùng."

ISP — sai và đúng

SAI — máy check-in bị ép cài cả method không dùng

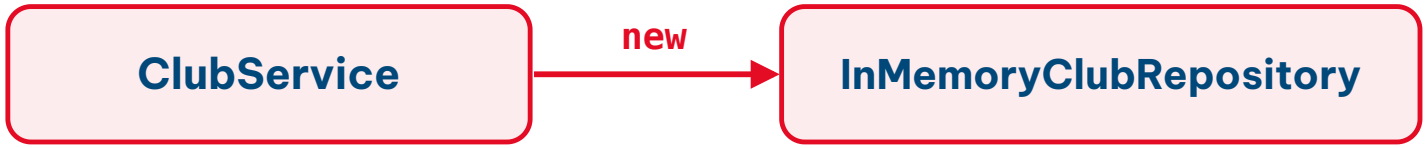
```
public interface IEventService {
  Event CreateEvent(...);   void Register(...);
  void CheckIn(...);        string ExportReport();
}
public class KioskCheckIn : IEventService {    // máy chỉ cần CheckIn
  public Event CreateEvent(...) => throw new NotImplementedException();
  /* Register, ExportReport: cũng ném NotImplementedException */
}
```

ĐÚNG — tách nhỏ, ai cần gì ký nấy

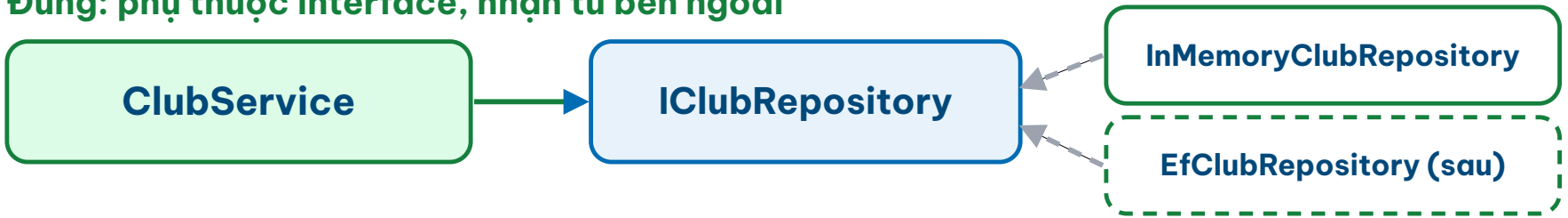
```
public interface IEventPlanner { Event CreateEvent(...); }
public interface IEventCheckIn { void CheckIn(Guid eventId, Guid memberId); }
public class KioskCheckIn : IEventCheckIn { /* chỉ cài CheckIn */ }
```

D – Dependency Inversion: dựa vào interface

Sai: tự new class cụ thể



Đúng: phụ thuộc interface, nhận từ bên ngoài



Hình 2.6. Class cấp cao (ClassService) không nên biết class cấp thấp cụ thể; cả hai cùng dựa vào interface (tự vẽ).

"Phụ thuộc vào abstraction, đừng phụ thuộc vào cài đặt cụ thể."

DIP – sai và đúng

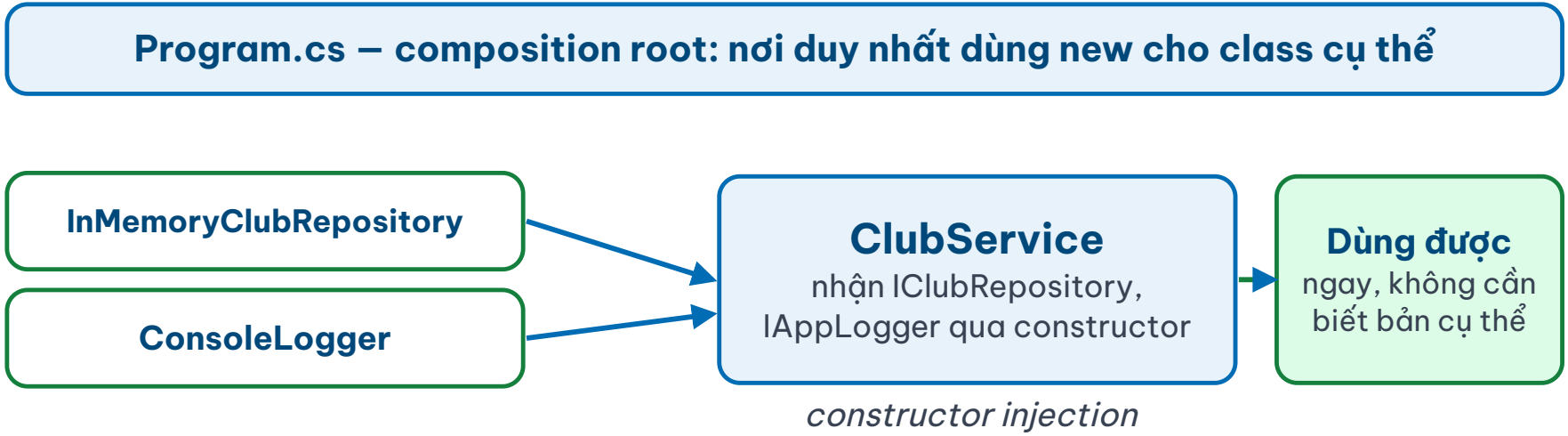
SAI – tự new bên trong: đổi nơi lưu dữ liệu hay viết test đều phải sửa class này

```
public class ClubService {
  private readonly InMemoryClubRepository _repo = new();
}
```

ĐÚNG – nhận interface qua constructor (Dependency Injection)

```
public class ClubService : IClubService {
  private readonly IClubRepository _clubs;
  public ClubService(IClubRepository clubs) => _clubs = clubs;  // đưa từ ngoài
vào
}
```

Dependency Injection: đưa thứ cần dùng từ bên ngoài vào



Hình 2.7. Chỉ Program.cs tạo các đối tượng cụ thể rồi đưa vào constructor của service (tự vẽ).

DI trong Program.cs của EduHub

```
IAppLogger logger = new ConsoleLogger();
INotificationService notifier = new EmailNotificationService();

IClubRepository clubRepo =
    new LoggingClubRepository(new InMemoryClubRepository(), logger);
IMemberRepository memberRepo = new InMemoryMemberRepository();

IClubService clubService =
    new ClubService(clubRepo, memberRepo, notifier, logger);
```

Đoạn mã 2.2. Composition root của đáp án mẫu.

- Ghi log ra file thay vì Console: đổi một dòng ở trên, các service không đổi
- Chương 3 sẽ giao việc "ráp" này cho DI container của .NET

2.1 SOLID trong EduHub: bài tập Ch02 sửa gì?

NGUYÊN LÝ	VI PHẠM TRONG CLUBMANAGER	CÁCH SỬA TRONG ĐÁP ÁN
S	CreateClub kiểm tra, lưu, log, báo tin	ClubService, IAppLogger, INotificationService
O	if/else theo Gmail, Yahoo	Mỗi kênh một class; thêm kênh không sửa code cũ
L, I	Một kho dữ liệu ôm mọi thao tác	Repository và service chia theo interface nhỏ
D	File.AppendAllText, Console.WriteLine trong logic	Gọi qua IAppLogger; Program.cs là nơi duy nhất new

Bảng 2.1. Đối chiếu vi phạm SOLID của starter và cách sửa trong đáp án.

Lab: tìm lỗi SOLID trong ClubManager

1. Mở ClubManager.cs trong starter của Bài tập Ch02
2. Ghi ra **ít nhất 4 vi phạm**, mỗi vi phạm theo mẫu: dòng nào, nguyên lý nào, hậu quả gì
3. Đề xuất cách sửa cho từng vi phạm (refactor: đổi cấu trúc code, chức năng giữ nguyên)
4. Trao đổi với bạn bên cạnh, rồi chia sẻ kết quả với lớp

> Mẹo: tìm class quá dài, chuỗi if/else dài, chỗ gọi **new** bên trong method, và chỗ gọi thẳng File hay Console.

2.2

PHẦN 2 / 5

MỤC 2.2 CỦA ĐỀ CƯƠNG

Bốn Design Pattern trọng tâm gắn với EduHub

- Design Pattern là gì · ba nhóm GoF
- Factory (Creational)
- Decorator (Structural)
- Strategy và Observer (Behavioral)
- Chạy đáp án mẫu

2.2 Design Pattern là gì? Ba nhóm của GoF



Creational

Cách **tạo** object
Factory, Builder,
Singleton



Structural

Cách **ghép** class lại
Decorator,
Adapter, Facade



Behavioral

Cách **chia việc**
giữa các object
Strategy, Observer,
Command

- Pattern là **cách giải bài toán thiết kế đã được kiểm chứng**, không phải thư viện để cài
- Nguồn: sách *Design Patterns* của nhóm GoF (Gamma và cộng sự, 1994), tài liệu [5] của học phần
- Màu đỏ là bốn pattern trọng tâm làm trong bài tập Ch02

2.2 Bốn pattern nằm ở đâu trong EduHub?

PATTERN	BÀI TOÁN CỦA EDUHUB	THÀNH PHẦN TRONG ĐÁP ÁN
Factory	Tạo đúng kênh thông báo theo yêu cầu	NotificationServiceFactory
Decorator	Thêm ghi log cho kho dữ liệu mà không sửa nó	LoggingClubRepository
Strategy	Xuất báo cáo theo nhiều định dạng	IReportStrategy, Console, CSV
Observer	Báo cho nhiều bên khi có sự kiện mới	IEventObserver, EmailNotifier, LogNotifier

Bảng 2.2. Bốn pattern trọng tâm và vị trí trong code đáp án mẫu.

2.2 Factory (Creational): "nhà máy" tạo object



Hình 2.8. Nơi gọi chỉ nói cần kênh nào; nhà máy tự tạo và trả về INotificationService (tự vẽ).

- Dùng khi **chỉ biết loại cần tạo lúc chạy**, hoặc cách tạo phức tạp
- Thay chuỗi if/else chọn kênh ở ClubManager

Factory — code (1/2): interface và nhà máy

```
public enum NotificationChannel { Email, Sms, Slack }

public interface INotificationServiceFactory {
  INotificationService Create(NotificationChannel channel);
}

public class NotificationServiceFactory : INotificationServiceFactory {
  public INotificationService Create(NotificationChannel c) => c switch {
    NotificationChannel.Email => new EmailNotificationService(),
    NotificationChannel.Sms   => new SmsNotificationService(),
    NotificationChannel.Slack => new SlackNotificationService(),
    _ => throw new ArgumentOutOfRangeException(nameof(c))
  };
}
```

Đoạn mã 2.3. Factory tạo kênh thông báo (rút gọn từ đáp án mẫu).

Factory — code (2/2): cách dùng

```
INotificationServiceFactory factory = new NotificationServiceFactory();

// Nơi gọi không new class cụ thể, chỉ truyền tên kênh
INotificationService notifier = factory.Create(NotificationChannel.Email);
notifier.Send("a@student.dthu.edu.vn", "Welcome!");

foreach (var channel in Enum.GetValues<NotificationChannel>())
    factory.Create(channel).Send("a@student.dthu.edu.vn", $"Thử {channel}");
```

Đoạn mã 2.4. Dùng Factory trong Program.cs của đáp án mẫu.

Thêm kênh mới: viết thêm class cài đặt INotificationService và thêm **một nhánh** trong nhà máy. Nơi gọi không đổi.

Decorator (Structural): "gói quà" thêm tính năng



Hình 2.9. Decorator cùng interface với class gốc và bọc nó lại để thêm ghi log (tự vẽ).

- Thêm hành vi **mà không sửa class gốc**, không cần kế thừa
- Bỏ decorator ở Program.cs thì dòng log biến mất, service không phải sửa

```
public class LoggingClubRepository : IClubRepository {
  private readonly IClubRepository _inner;     // repository thật bên trong
  private readonly IAppLogger _logger;
  /* constructor nhận _inner và _logger */

  public void Add(Club club) {
    _logger.Info($"[repo] Add club '{club.Name}'");  // việc thêm vào
    _inner.Add(club);                                 // giao cho bản thật
  }
  /* GetAll, GetById: ghi log rồi giao lại tương tự */
}
```

Đoạn mã 2.5. Decorator ghi log cho kho dữ liệu (rút gọn từ đáp án mẫu).

Program.cs bọc bên ngoài: new LoggingClubRepository(repo, logger).

2.2 Strategy (Behavioral): đổi "cách làm"



Hình 2.10. Cùng một danh sách thống kê, mỗi strategy vẽ ra một định dạng khác (tự vẽ).

- Mỗi cách làm là một class riêng, **đổi được lúc chạy**
- Thêm định dạng mới (PDF, JSON) không phải sửa ReportService: đúng OCP

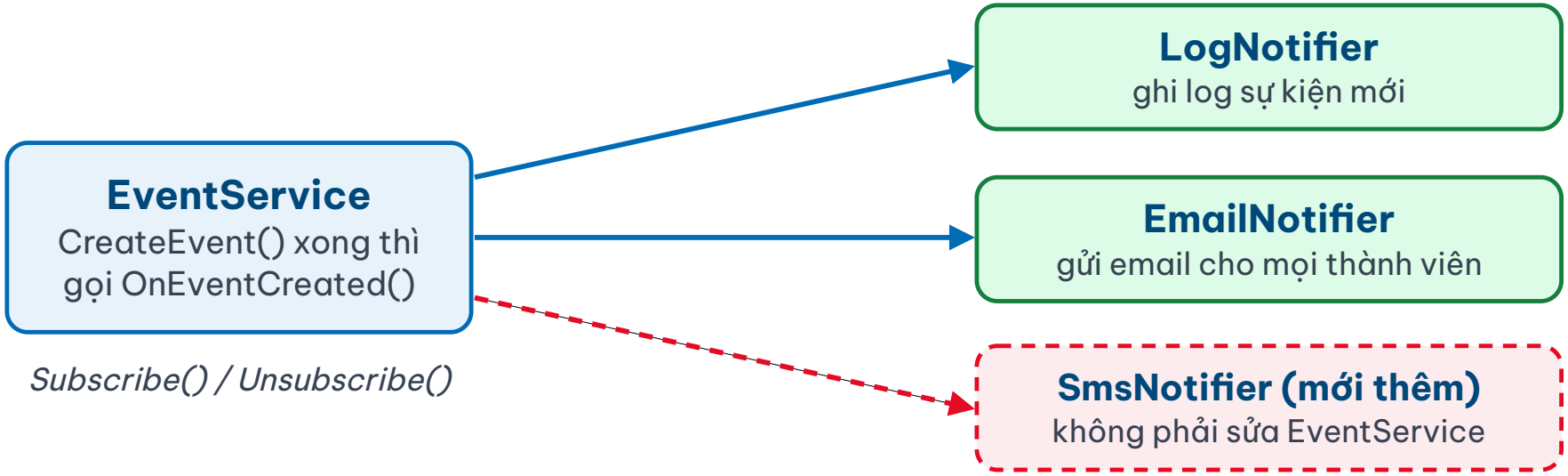
```
public interface IReportStrategy {
  string Name { get; }
  string Render(IReadOnlyList<ClubStat> stats);
}

public class CsvReportStrategy : IReportStrategy {
  public string Name => "CSV";
  public string Render(IReadOnlyList<ClubStat> stats) =>
    "Club,Members\n" + string.Join("\n",
      stats.Select(s => $"{s.ClubName},{s.Members}"));
}

// ReportService: return strategy.Render(stats);   Nơi dùng:
string csv = reportService.Generate(new CsvReportStrategy());
```

Đoạn mã 2.6. Strategy xuất CSV (rút gọn từ đáp án mẫu).

2.2 Observer (Behavioral): "bấm chuông" nhận tin



Hình 2.11. EventService báo cho mọi observer đã đăng ký mà không cần biết họ làm gì (tự vẽ).

- Một object đổi trạng thái thì **mọi bên đã đăng ký được báo tự động**
- Giống bấm chuông kênh YouTube: thêm người theo dõi không đổi kênh

Observer — code

```
public interface IEventObserver {
  void OnEventCreated(Event ev, Club club);     // gọi khi có sự kiện mới
}

// EventService là "người phát tin"
private readonly List<IEventObserver> _observers = new();
public void Subscribe(IEventObserver o) => _observers.Add(o);

public Event CreateEvent(Guid clubId, string title, DateTime date) {
  /* ... tạo và lưu sự kiện ... */
  foreach (var o in _observers.ToList()) o.OnEventCreated(ev, club);
  return ev;
}
```

Đoạn mã 2.7. Observer báo sự kiện mới (rút gọn từ đáp án mẫu).

Đăng ký người nghe: eventService.Subscribe(new LogNotifier(logger)).

2.2 Chọn pattern nào khi nào?

DẤU HIỆU TRONG CODE	PATTERN
Nhiều *if/else* hoặc *switch* chỉ để quyết định tạo class nào	**Factory**
Muốn thêm hành vi (log, cache) mà không sửa class gốc	**Decorator**
Cùng một việc nhưng có nhiều cách làm, đổi được lúc chạy	**Strategy**
Một thay đổi phải báo cho nhiều bên, số bên có thể tăng	**Observer**

Hỏi trước khi dùng: **"Pattern này giải quyết vấn đề gì của mình?"** Chưa có vấn đề thì chưa cần pattern.

Bảng 2.3. Dấu hiệu nhận biết để chọn pattern.

Demo: chạy đáp án mẫu

Mở thư mục đáp án của giảng viên và chạy

```
cd 02_Projects/ch02/solution
dotnet run --project EduHub.Ch02
dotnet test
```

- Mở **Program.cs** trước: đây là nơi duy nhất dùng new để ráp các class lại
- Chạy xong có 5 khối: tạo CLB, Observer, Strategy, Factory, kiểm tra dữ liệu sai
- **[repo]** do Decorator ghi; **[LOG]** và **[EMAIL]** do hai observer

Demo: đọc kết quả chạy

```
== 2. Observer: tạo sự kiện -> 2 observer được báo ==
  [LOG] Event created: 'Workshop SOLID' (Tech Club)
  [EMAIL] to a@student.dthu.edu.vn: New event 'Workshop SOLID' ...

== 3. Strategy: báo cáo CSV ==
Club,Members,Events,Registered,CheckedIn
"Tech Club",2,1,2,1

== 4. Factory: tạo kênh thông báo theo yêu cầu ==
  [EMAIL] to a@student.dthu.edu.vn: Thử kênh Email
  [SMS] to a@student.dthu.edu.vn: Thử kênh Sms
```

- Muốn gửi thêm SMS khi có sự kiện mới, em viết gì và sửa file nào?
- Bỏ Decorator ở Program.cs thì service có phải sửa không?

2.3

PHẦN 3 / 5

MỤC 2.3 CỦA ĐỀ CƯƠNG

Các pattern còn lại và pattern có sẵn trong .NET

- Builder, Singleton, Adapter, Facade, Command
- Pattern trong .NET: DI, middleware, Options
- Tránh lạm dụng pattern

2.3 Các pattern còn lại: nhìn nhanh

Builder	Singleton	Adapter	Facade	Command
Creational	Creational	Structural	Structural	Behavioral
Dựng object phức tạp từng bước	Chỉ một object dùng chung	Đổi giao diện cho khớp nhau	Một cửa gọn cho nhiều class	Đóng gói một yêu cầu thành object

- Chỉ cần **nhận ra và biết dùng khi cần**; bài tập Ch02 không bắt buộc cài đặt các pattern này
- Các ví dụ tiếp theo là minh hoạ gắn với EduHub

Builder: dựng object từng bước

```
var ev = new EventBuilder().ForClub(club.Id)
    .WithTitle("Workshop SOLID")
    .OnDate(new DateTime(2026, 11, 20))
    .Build();                    // kiểm tra đủ dữ liệu rồi mới tạo Event

public class EventBuilder {
  private Guid _clubId; private string? _title; private DateTime _date;
  public EventBuilder WithTitle(string t) { _title = t; return this; }
  public EventBuilder OnDate(DateTime d)  { _date = d;  return this; }
  public Event Build() => new() { ClubId = _clubId, Date = _date,
    Title = _title ?? throw new InvalidOperationException("Title required") };
}
```

Đoạn mã 2.8. Builder cho Event (minh hoạ, không có trong đáp án).

2.3 Singleton: một object dùng chung (và lời cảnh báo)

```
// Cách cũ: static Instance, ai cũng gọi thẳng, khó thay bản giả khi test
public class AppConfig {
  public static AppConfig Instance { get; } = new();
  private AppConfig() { }
}

// Cách trong .NET: để DI container giữ đúng một object
builder.Services.AddSingleton<IClubRepository, InMemoryClubRepository>();
```

Đoạn mã 2.9. Singleton kiểu static và Singleton do DI quản lý.

Lạm dụng Singleton kiểu static khiến code **dính chặt vào nhau và khó kiểm thử**. Ưu tiên để DI quản lý vòng đời.

```
// Thư viện SMS bên ngoài có API riêng, không khớp INotificationService
public class ExternalSmsClient { public bool SendSms(string phone, string text); }

public class SmsGatewayAdapter : INotificationService {
  private readonly ExternalSmsClient _client;
  public SmsGatewayAdapter(ExternalSmsClient c) => _client = c;

  public void Send(string to, string message)
    => _client.SendSms(to, message);      // dịch sang API bên ngoài
}
```

Đoạn mã 2.10. Adapter đưa thư viện ngoài về INotificationService.

- Code của ta vẫn chỉ biết INotificationService; thư viện lạ được "bọc" lại

```
public class EventRegistrationFacade {
  private readonly IEventService _events;
  private readonly IAppLogger _logger;
  private readonly INotificationService _notifier;
  /* constructor nhận ba phụ thuộc */

  public void RegisterAndConfirm(Guid eventId, Guid memberId, string email) {
    _events.Register(eventId, memberId);              // 1. đăng ký
    _logger.Info($"Member {memberId} registered");    // 2. ghi log
    _notifier.Send(email, "Đăng ký thành công!");     // 3. báo thành viên
  }
}
```

Đoạn mã 2.11. Facade gom ba bước thành một lời gọi (minh hoạ).

- Nơi gọi chỉ cần một dòng; chi tiết bên trong được giấu đi

```
public interface ICommand { void Execute(); void Undo(); }

public class AddMemberCommand : ICommand {
  private readonly IClubService _service; private readonly Guid _clubId;
  private readonly string _name, _email;
  public void Execute() => _service.AddMember(_clubId, _name, _email);
  public void Undo()    { /* gỡ thành viên vừa thêm */ }
}

var history = new Stack<ICommand>();
cmd.Execute(); history.Push(cmd);       // làm và nhớ lại
history.Pop().Undo();                   // hoàn tác lệnh gần nhất
```

Đoạn mã 2.12. Command cho phép hoàn tác thao tác (minh hoạ).

BẠN GẶP TRONG .NET	LÀ PATTERN GÌ
Dependency Injection (builder.Services.AddScoped...)	Nền tảng DIP; container tạo và cấp object
Middleware (app.Use...)	Chuỗi xử lý nối tiếp (Chain of Responsibility)
Options (IOptions<T>, Configure<T>)	Cấu hình thành class có kiểu, nhận qua DI
WebApplication.CreateBuilder()	Builder
AddSingleton	Singleton do container quản lý

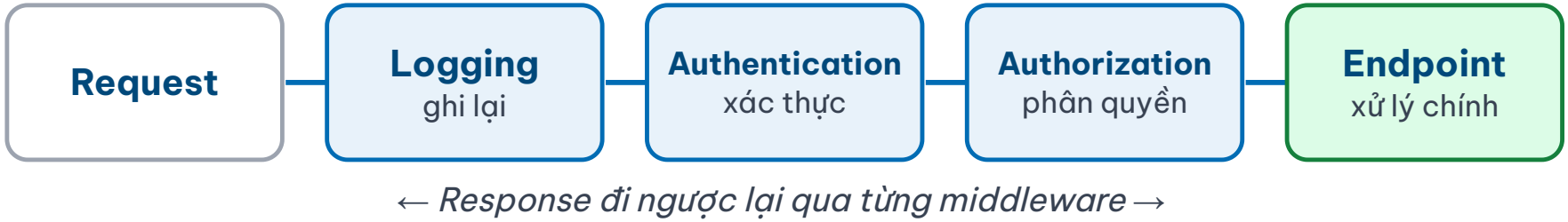
Bảng 2.4. Pattern có sẵn trong .NET.

```
// "Ai cần IClubRepository thì đưa InMemoryClubRepository"
builder.Services.AddSingleton<IClubRepository, InMemoryClubRepository>();
// ^ Singleton: một bản dùng chung cho cả ứng dụng
builder.Services.AddScoped<IClubService, ClubService>();   // mỗi request một bản
builder.Services.AddTransient<INotificationService,
    EmailNotificationService>();                           // mỗi lần xin một bản

// ClubService chỉ khai báo cần gì ở constructor, container tự cấp
public class ClubService(IClubRepository clubs) : IClubService { /* ... */ }
```

Đoạn mã 2.13. Đăng ký và nhận phụ thuộc bằng DI container.

Giống Program.cs thủ công ở mục 2.1, nhưng container làm thay. Học kỹ ở Chương 3.



Hình 2.12. Mỗi middleware xử lý rồi chuyển cho middleware kế tiếp (Chain of Responsibility; tự vẽ).

```
app.Use(async (context, next) => {
    Console.WriteLine($"-> {context.Request.Path}");  // trước khi chuyển tiếp
    await next();                                      // chuyển cho middleware kế
    Console.WriteLine($"<- {context.Response.StatusCode}");  // sau khi có kết quả
});
```

```
// appsettings.json:  "Club": { "MaxMembers": 50 }
public class ClubOptions { public int MaxMembers { get; set; } = 50; }

// Program.cs: đọc mục "Club" trong cấu hình rồi đăng ký
builder.Services.Configure<ClubOptions>(
    builder.Configuration.GetSection("Club"));

// Nơi dùng: nhận cấu hình qua constructor, không đọc file thủ công
public class ClubService(IOptions<ClubOptions> options) {
  private readonly int _max = options.Value.MaxMembers;
}
```

Đoạn mã 2.14. Options thay hằng số 50 cứng trong code.

- Đổi giới hạn thành viên chỉ cần sửa cấu hình, không build lại

2.3 Đừng lạm dụng pattern



God Class

Một class ôm mọi việc, như ClubManager. Đây là thứ ta đang sửa.



Lạm dụng Singleton

Mọi thứ đều static, dùng chung. Khó test và dễ lỗi khi nhiều luồng.



Cargo cult

Bắt chước pattern mà không hiểu. Ví dụ dùng Factory khi chỉ có một loại để tạo.

Quy tắc: **gặp vấn đề rồi mới chọn pattern**, không đi tìm chỗ để nhét pattern.

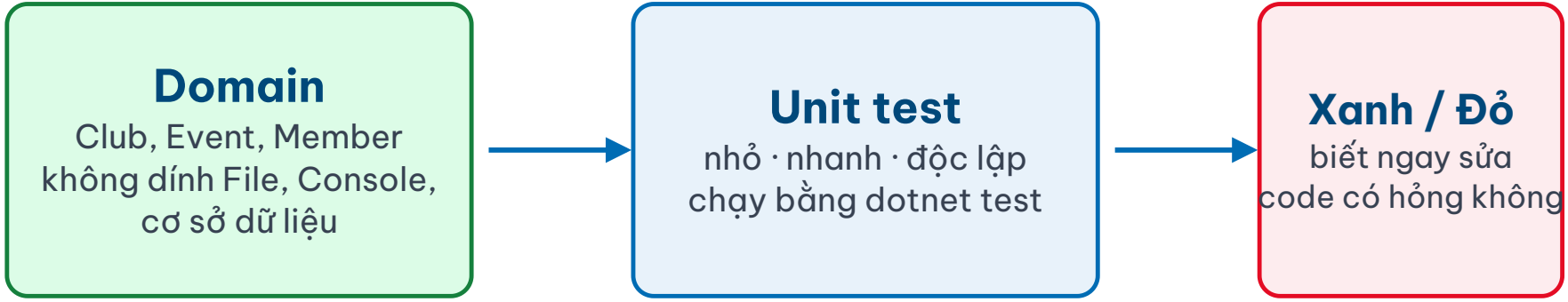
2.4

PHẦN 4 / 5

MỤC 2.4 CỦA ĐỀ CƯƠNG

Unit test đầu tiên cho lớp Domain

- Vì sao cần unit test
- xUnit và dotnet test
- Arrange-Act-Assert
- Test trường hợp lỗi và giá trị biên



Hình 2.13. Unit test chạy trên lớp Domain thuần, không cần mạng hay cơ sở dữ liệu (tự vẽ).

- Domain chứa luật nghiệp vụ nên là nơi **đáng kiểm thử nhất** và dễ kiểm thử nhất
- Nhờ SOLID/DIP ở mục 2.1, các lớp phụ thuộc interface nên cũng thay được bằng bản giả

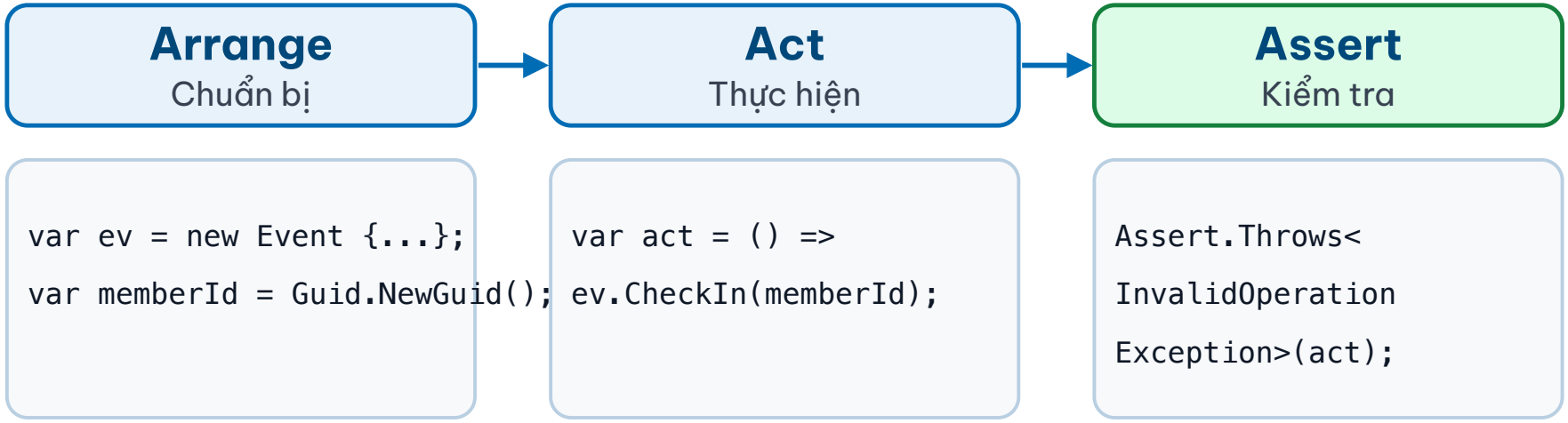
2.4 xUnit: tạo project test

```
dotnet new xunit -n EduHub.Ch02.Tests          # tạo project test
dotnet add EduHub.Ch02.Tests reference EduHub.Ch02  # tham chiếu project
dotnet sln add EduHub.Ch02.Tests               # thêm vào solution
dotnet test                                    # chạy mọi test
```

```
solution/
├── EduHub.Ch02/
│   └── Domain/ Club.cs Event.cs
└── EduHub.Ch02.Tests/
    └── Domain/ ClubTests.cs
                EventTests.cs
```

- Mỗi lớp cần kiểm thử có một file **XxxTests.cs**
- Method test gắn **[Fact]**; chạy sai thì test đỏ

Arrange-Act-Assert: ba phần của một test



Hình 2.14. Mỗi test chia thành chuẩn bị, thực hiện, kiểm tra; ví dụ: check-in khi chưa đăng ký phải báo lỗi (tự vẽ).

- Mỗi test kiểm tra **một hành vi**, tách ba phần bằng dòng trống hoặc chú thích

Test đầu tiên cho Club.Create

```
[Fact]
public void Create_WithValidName_ReturnsClubWithNameAndNewId() {
  // Arrange
  const string name = "Tech Club";
  // Act
  var club = Club.Create(name, "Câu lạc bộ công nghệ");
  // Assert
  Assert.Equal(name, club.Name);
  Assert.NotEqual(Guid.Empty, club.Id);
}
```

Đoạn mã 2.15. Test AAA đầu tiên (ClubTests.cs).

Tên test: Method_TìnhHuống_KếtQuả.

Một test, nhiều dữ liệu: [Theory] và giá trị biên

```
[Theory]
[InlineData(0, true)]
[InlineData(49, true)]
[InlineData(50, false)]                    // biên: đủ 50 thì hết chỗ
public void HasRoom_DependsOnCurrentMemberCount(int count, bool expected) {
  var club = Club.Create("Tech Club", "mô tả");   // Arrange
  var result = club.HasRoom(count);                // Act
  Assert.Equal(expected, result);                  // Assert
}
```

Đoạn mã 2.16. Theory chạy một test với nhiều bộ dữ liệu, gồm cả biên.

- Luôn thử các **giá trị biên**: 0, sát giới hạn, đúng giới hạn

Test trường hợp lỗi: Assert.Throws

```
[Fact]
public void CheckIn_UnregisteredMember_ThrowsInvalidOperationException() {
  // Arrange
  var ev = new Event { Title = "Workshop SOLID" };

  // Act
  var act = () => ev.CheckIn(Guid.NewGuid());   // chưa đăng ký mà check-in

  // Assert
  Assert.Throws<InvalidOperationException>(act);
}
```

Đoạn mã 2.17. Luật nghiệp vụ: chỉ người đã đăng ký mới được check-in.

- Test tốt phủ cả **đường đúng** lẫn **đường lỗi**

Chạy test và đọc kết quả

```
$ dotnet test
Passed!  - Failed: 0, Passed: 21, Skipped: 0, Total: 21
```

Test tốt

- Nhanh, độc lập, chạy thứ tự nào cũng được
- Một hành vi mỗi test
- Tên nói rõ tình huống và kết quả

Tránh

- Test chỉ để tăng con số coverage
- Nhiều Assert không liên quan trong một test
- Phụ thuộc file, mạng, giờ hệ thống

2.5

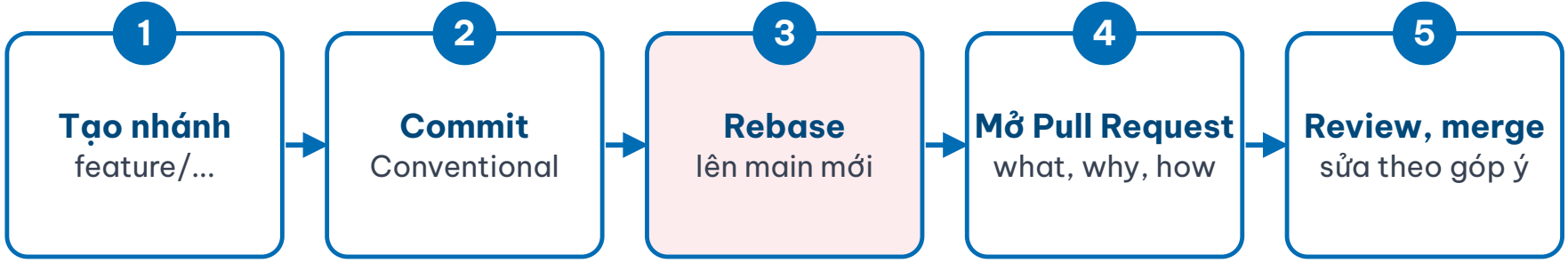
PHẦN 5 / 5

MỤC 2.5 CỦA ĐỀ CƯƠNG

Git workflow nâng cao

- Quy trình nhóm: nhánh, rebase, Pull Request
- Rebase và xử lý conflict
- Code review trên Pull Request

2.5 Quy trình làm việc nhóm trên GitHub

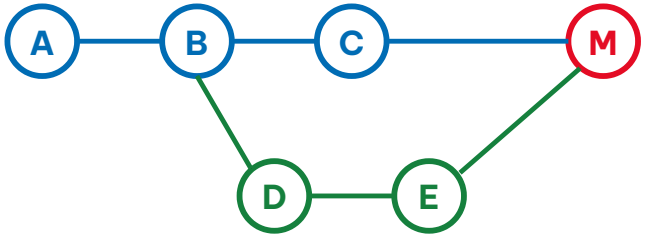


Hình 2.15. Năm bước từ nhánh riêng đến khi gộp vào main; Chương 2 thêm bước rebase và review (tự vẽ).

- Không commit thẳng vào **main**: mọi thay đổi đi qua Pull Request
- Chương 1 đã học bước 1, 2, 4; hôm nay học bước **3 (rebase)** và **5 (review)**

2.5 Merge và rebase khác nhau thế nào?

Merge: thêm merge commit M



main: A, B, C, M · feature: D, E

Rebase: lịch sử thẳng một đường



D′, E′ là bản sao của D, E, đặt lên sau C
(mã định danh commit đổi)

```
git rebase origin/main
```

Hình 2.16. Cùng kết quả code, hai kiểu lịch sử (tự vẽ).

Rebase giúp lịch sử PR gọn, dễ đọc. Chỉ rebase nhánh **của riêng bạn**.

2.5 Rebase nhánh feature lên main mới nhất

```
git fetch origin                        # lấy main mới nhất về máy
git switch feature/solid-refactor       # đứng ở nhánh của mình
git rebase origin/main                  # đặt commit của mình lên đầu main mới

git push --force-with-lease             # đẩy lại (lịch sử đã đổi)
```

Đoạn mã 2.18. Rebase trước khi mở hoặc cập nhật Pull Request.

Rebase **viết lại lịch sử**. Không rebase nhánh mà người khác đang dùng; luôn dùng **--force-with-lease**, không dùng --force trần.

Conflict: hai người sửa cùng một chỗ

Git dừng lại và đánh dấu trong file, bạn là người quyết định giữ phần nào

```
<<<<<<< HEAD
    public const int MaxMembers = 60;      // bản của main
=======
    public const int MaxMembers = 50;      // bản của nhánh feature
>>>>>>> feature/solid-refactor
```

Đoạn mã 2.19. Dấu hiệu conflict trong file Club.cs.

- Phần giữa `<<<<<<<` và `=======` là bản đã có trên main
- Phần giữa `=======` và `>>>>>>>` là bản của bạn

2.5 Xử lý conflict: năm bước

1	**Mở file bị conflict**	git status cho biết file nào
2	**Chọn nội dung đúng**	giữ một bản hoặc trộn hai bản
3	**Xoá dấu <<<, ===, >>>**	file phải build và test được
4	**git add file**	báo đã giải quyết xong
5	**git rebase --continue**	hoặc --abort để huỷ làm lại

Hình 2.17. Quy trình giải quyết conflict khi rebase (tự vẽ).

- Sau khi giải quyết xong, chạy lại **dotnet build** và **dotnet test** trước khi push
- Ghi cách xử lý vào mô tả Pull Request

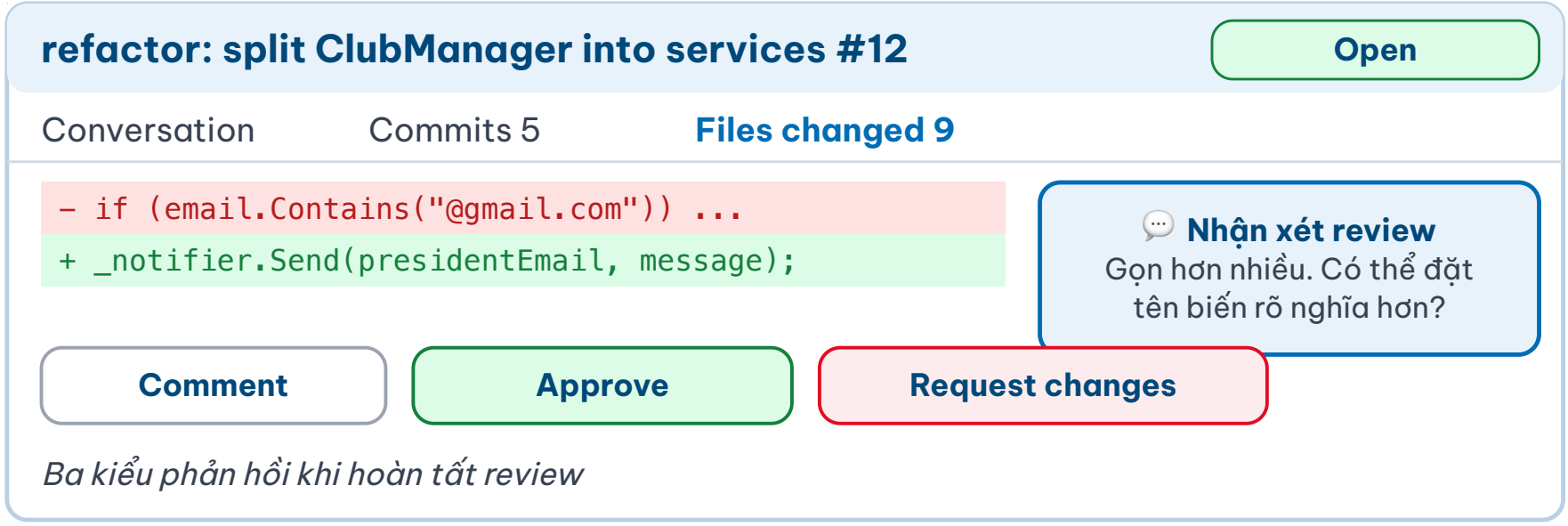
Commit gọn trước khi mở Pull Request

```
feat(notify): add notification factory
fix(club): reject names over 100 chars
refactor(repo): extract IClubRepository
test(event): add check-in tests
docs(readme): add class diagram
```

- Mỗi commit là **một thay đổi có ý nghĩa**
- Dạng: loại(phạm vi): mô tả ngắn, thể mệnh lệnh
- Loại hay dùng: feat, fix, refactor, test, docs, chore

```
git rebase -i HEAD~3    # gộp (squash) hoặc đổi tên commit lộn xộn
```

Đoạn mã 2.20. Lịch sử commit theo Conventional Commits.



Hình 2.18. Minh hoạ trang Pull Request trên GitHub (tự vẽ).

- Reviewer đọc diff, **comment theo dòng**, rồi Approve hoặc Request changes

```
## What
Tách ClubManager thành ClubService,
EventService, ReportService.

## Why
ClubManager vi phạm SRP, OCP, DIP.

## How to test
dotnet build
dotnet test
dotnet run --project EduHub.Ch02
```

- **Người viết:** mô tả đủ what, why, how to test; PR nhỏ, dễ đọc
- **Người review:** góp ý về code, không về con người; nêu lý do và gợi ý cách sửa
- Phân biệt lỗi phải sửa với gợi ý cải thiện
- Sửa xong thì push commit mới và trả lời từng nhận xét

Đoạn mã 2.21. Mẫu mô tả Pull Request cho Bài tập Ch02.

Lab: nộp Bài tập Ch02 bằng Pull Request

1. Tạo nhánh **feature/solid-refactor**, làm từng nhiệm vụ và commit theo Conventional Commits (ít nhất 5 commit)
2. Chạy **dotnet build** và **dotnet test** cho tới khi xanh
3. **Rebase** lên main mới nhất, tự xử lý conflict nếu có
4. Mở Pull Request với mô tả what, why, how to test; để lại ít nhất 2 nhận xét review ngay trên PR của mình
5. Gửi link Pull Request theo hướng dẫn trên LMS

Bài tập Ch02 — Refactoring EduHub

NHIỆM VỤ	NỘI DUNG	ĐIỂM
1	Phân tích ít nhất 4 vi phạm SOLID trong ClubManager	15
2	Refactor thành tối thiểu 4 class, phụ thuộc qua interface	30
3	Factory, Decorator, Strategy, Observer	25
4	Từ 6 unit test xUnit theo Arrange-Act-Assert cho Domain	15
5	Nhánh, 5 commit trở lên, rebase, Pull Request có mô tả	15

Thuộc hoạt động đánh giá HĐ1.3 (bài tập chương), chấm theo rubric trong 02_Projects/ch02. Starter có sẵn các đoạn TODO.

Tổng kết Chương 2

NỘI DUNG	CHUẨN ĐẦU RA
SOLID; Factory, Decorator, Strategy, Observer; pattern trong .NET	**CLO1**
Git: rebase, conflict, Pull Request	**CLO2**
Refactor EduHub, unit test cho Domain	**CLO4**
Review code, góp ý và nhận phản hồi trên Pull Request	**CLO5**

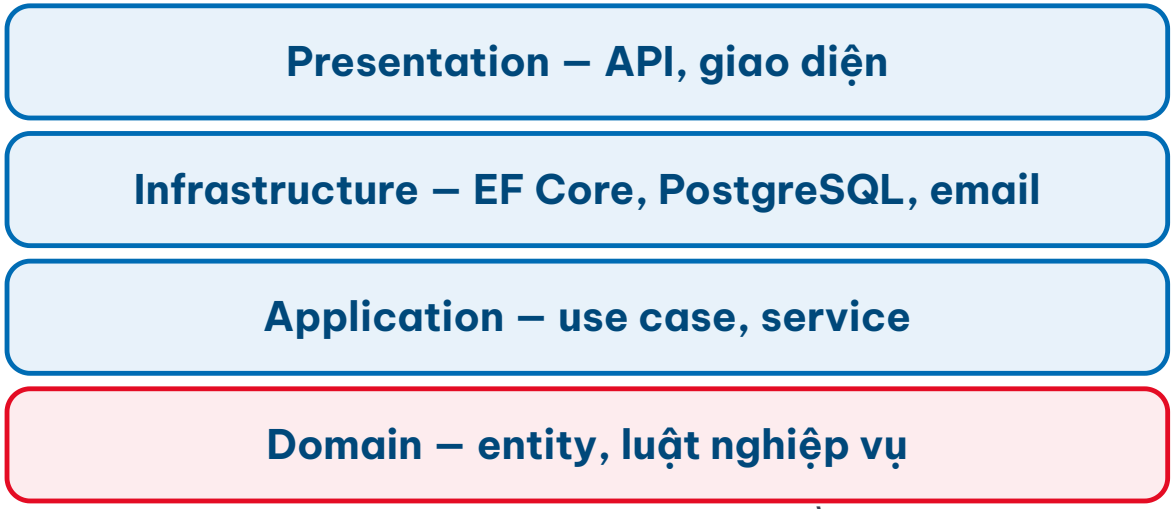
- Hiểu vấn đề trước, chọn pattern sau
- Từ Chương 3, sinh viên nghiên cứu và thuyết trình theo nhóm; giảng viên phản biện

Câu hỏi tự kiểm tra

✔ CHECKPOINT — CHƯƠNG 2

1. God class vi phạm nguyên lý nào trong SOLID? Chỉ ra một chỗ trong ClubManager.
2. Vì sao dùng interface và DI giúp unit test dễ hơn?
3. Muốn thêm kênh thông báo Slack, bạn thêm gì và sửa gì (Factory, OCP)?
4. Decorator khác kế thừa ở điểm nào? Observer khác Strategy ở điểm nào?
5. Rebase khác merge ở đâu? Khi nào không nên rebase?

Xem trước Chương 3: Clean Architecture và API



Hình 2.19. Chương 3 xếp SOLID và pattern vào kiến trúc bốn lớp (tự vẽ).

- Dependency Inversion và DI container của .NET; Controller-based API, Swagger, mã trạng thái HTTP
- Unit test cho lớp Application với Moq

Chuẩn bị cho Chương 3

📖 Đọc tài liệu [1] .NET 10 Documentation, [2] *Clean Architecture* và [3] *ASP.NET Core in Action*

🎤 Chương 3 trở đi, nhóm nghiên cứu và thuyết trình: chuẩn bị slide, nộp qua Pull Request

💻 Hoàn thiện Bài tập Ch02 và nộp Pull Request đúng hạn trên LMS

🌿 Luyện lại quy trình: nhánh riêng, commit rõ nghĩa, rebase, Pull Request